

Demostración y Complejidad del Algoritmo de Prim con Cola con Prioridades

Este documento presenta la demostración de la implementación óptima del algoritmo de Prim usando una cola con prioridades y la prueba de su complejidad temporal en $\Theta(a \log n)$.

1. Implementación de Prim mediante una Cola con Prioridades

Para evitar la búsqueda secuencial de coste $O(n)$ en cada paso para encontrar la arista de menor peso que conecta con un nodo fuera del Árbol de Recubrimiento Mínimo (MST), se utiliza una cola con prioridades (min-heap) Q que almacena los vértices restantes priorizados por su distancia mínima al árbol actual.

Estructuras de Datos

- `dist` (Arreglo de tamaño n): Almacena el peso mínimo de la arista desde cada vértice al MST actual (inicializado en ∞ , raíz en 0).
- `parent` (Arreglo de tamaño n): Almacena el nodo padre de cada vértice en el MST.
- `en_MST` (Arreglo booleano de tamaño n): Indica si un vértice ya forma parte del MST.
- Q (Min-Heap): Contiene elementos de la forma $(\text{dist}[v], v)$. El min-heap mantiene en la raíz al nodo v con el menor valor en `dist`.

2. Demostración de la Complejidad Temporal $\Theta(a \log n)$

Asumimos un grafo conexo $G = (V, E)$ donde $a \geq n - 1$, representado mediante una lista de adyacencia. Analizamos el coste de cada operación empleando un min-heap binario:

Paso 1: Inicialización

- Inicializar los arreglos `dist`, `parent` y `en_MST` toma un tiempo de $O(n)$.
- Insertar la raíz en el min-heap toma $O(\log n)$.
- Coste de inicialización: $O(n)$.

Paso 2: Análisis del Bucle Principal (Extracciones y Relajaciones)

A) Operaciones de Extracción (`ExtraerMinimo`)

- El algoritmo realiza exactamente una operación de extracción exitosa por cada uno de los n vértices.
- En una implementación *lazy* (perezosa sin decrecer clave explícitamente), también realiza extracciones redundantes de nodos ya procesados. El número total de inserciones y extracciones de la cola está acotado por el número total de aristas a .
- Como el tamaño máximo de la cola está acotado por $a \leq n^2$, cualquier extracción toma $O(\log a) = O(\log n)$.

- Coste acumulado de extracciones:

$$O(a \cdot \log n)$$

B) Bucle de Relajación de Vecinos

- Para cada vértice u extraído, recorremos sus arcos adyacentes.
- A lo largo de todo el algoritmo, **cada arista se examina exactamente dos veces** (una por cada extremo).
- El número total de comprobaciones de vecinos es exactamente $2a$.
- Coste acumulado de comprobación: $O(a)$.

C) Operaciones de Inserción / Actualización

- Si al evaluar una arista se encuentra un camino más corto hacia un vértice v , se inserta su nuevo coste en el heap.
- El número máximo de estas inserciones está acotado por el número de aristas a .
- Cada inserción en el min-heap toma un tiempo de $O(\log n)$.
- Coste acumulado de inserciones:

$$O(a \cdot \log n)$$

3. Coste Total Ajustado

Sumando todos los costes del algoritmo:

$$T(n, a) = O(n) + \text{Extracciones} + \text{Inserciones}$$

$$T(n, a) = O(n) + O(a \cdot \log n) + O(a \cdot \log n) = O(a \cdot \log n)$$

Dado que el grafo es conexo, se cumple que $a \geq n - 1$. Esto implica que el término $a \cdot \log n$ domina asintóticamente a n . Como no hay atajos que permitan evitar procesar las aristas o las actualizaciones en el heap para verificar el MST, se demuestra que la cota es ajustada:

$$T(n, a) \in \Theta(a \cdot \log n)$$